**UNIVERSIDAD SANTO TOMÁS**
Facultad de Ingeniería en Tecnologías de la Información y las Comunicaciones · Ingeniería en Informática

# Práctica de Integración Hardware, Software y Firmware (HSF)

*Prototipo de captura, validación y análisis de telemetría de un dispositivo, integrado con el sistema operativo*



| | |
|---|---|
| **Estudiante** | Jhojann Sthebeen Aristizabal Cruz |
| **Espacio académico** | Integración de Sistemas de Información (74271, plan 4, nivel 9) |
| **Docente** | Crisman Martínez B. |
| **Entrega** | Segundo corte · 11 de octubre de 2026 |



## Resumen

Este documento describe el diseño, la implementación, las pruebas y los resultados de un prototipo de sistema de información que recorre de extremo a extremo el camino **hardware → firmware → comunicación → sistema operativo → aplicación Python → persistencia y reporte → usuario**. Como no se dispone de una placa física, el firmware se simula con un dispositivo virtual (ENV-001) que emite telemetría ambiental (temperatura, humedad y voltaje) por TCP, con identificación, numeración de muestras, control de integridad CRC32 y capacidad de inyectar anomalías y fallos de comunicación.

La aplicación Python es modular (adquisición, validación, lógica, persistencia, reporte), consulta recursos del sistema operativo, registra eventos con niveles de log, guarda los datos en CSV y genera un reporte con indicadores técnicos. Se ejecutaron 9 pruebas automatizadas (9 aprobadas) y cuatro tipos de fallos controlados. En operación normal se cumplieron todas las metas (100 % de disponibilidad, 100 % de datos válidos, 0 % de errores). Con fallos inyectados el sistema no se cerró y se recuperó en 3,0 s, aunque las metas de calidad no se cumplieron, lo cual era esperable y se analiza en la sección 8.

**Palabras clave:** integración hardware-software, firmware, telemetría, sistema operativo, Python, tolerancia a fallos, observabilidad.

## 1. Análisis del problema (Fase 1)

**Caso de uso.** Una organización necesita un prototipo que capture datos de un sensor ambiental de un cuarto técnico, los procese y genere información para decidir. Se debe alertar cuando la temperatura supere 35 °C, la humedad supere 85 % o el voltaje baje de 3,0 V.

| Elemento | Definición |
|---|---|
| **Entradas** | Trama JSON por TCP (una línea por muestra): device_id, seq, ts, temp_c, hum_pct, volt_v, crc. |
| **Salidas** | data.csv (datos), events.log (eventos), report.md y report.json (resumen), estado de integración en consola. |
| **Actores** | Dispositivo ENV-001 (simulado), aplicación Python, operador que consulta el reporte. |
| **Restricciones** | Solo biblioteca estándar de Python más psutil; sin ejecutar comandos del sistema; ejecución local (127.0.0.1). |
| **Criterios de aceptación** | Disponibilidad de comunicación ≥ 95 %; datos válidos ≥ 98 %; tasa de errores ≤ 5 %; recuperación ≤ 5 s. |

### Asignación de componentes a capas.

| Capa | Responsabilidad | En este prototipo | Evidencia |
|---|---|---|---|
| **Hardware** | Captura señales físicas | Sensor de temperatura, humedad y voltaje (simulado) | Valores generados con ruido gaussiano |
| **Firmware** | Control de bajo nivel y protocolo | firmware_sim/device_sim.py | Trama JSON con CRC32 |
| **Sistema operativo** | Procesos, memoria, archivos, red, permisos | Linux/Windows: sockets, archivos, CPU, memoria, disco | Conexión TCP, archivos, métricas |
| **Software** | Procesa datos y ofrece funcionalidad | Paquete app/ en Python | Logs, CSV y reporte |

## 2. Diseño de la arquitectura (Fase 2)




*Transformación: señal → valor muestreado → trama con CRC → bytes TCP → objeto validado → fila CSV → reporte*

*Figura 1. Arquitectura de capas y flujo de datos (fuente: elaboración propia).*

**Interfaces y protocolo.** El dispositivo actúa como servidor TCP en 127.0.0.1 y la aplicación como cliente. Cada muestra es una línea JSON en UTF-8 terminada en salto de línea. El campo *crc* es el CRC32 del payload serializado con claves ordenadas y sin el propio campo:

```
{"device_id": "ENV-001", "seq": 42, "ts": 1791732053.8097327, "temp_c": 23.69,
 "hum_pct": 57.05, "volt_v": 3.691, "crc": "1db0fa5c"}
```

| Decisión | Valor | Justificación |
|---|---|---|
| **Frecuencia de lectura** | 10 muestras/s (20/s en la demostración) | Suficiente para monitoreo ambiental; deja margen de CPU |
| **Timeout de lectura** | 1,0 s; 2 timeouts seguidos = enlace perdido | Evita falsos positivos por un retraso aislado |
| **Reconexión** | Espera 0,2 s que se duplica hasta 2 s | Reintenta rápido al inicio sin saturar el puerto |
| **Límites de entrada** | Trama ≤ 4 KB; buffer ≤ 64 KB | Defensa ante tramas gigantes (denegación de servicio) |
| **Almacenamiento** | CSV por ejecución, con buffer en memoria (≤ 1000 filas) | Simple de auditar; tolera fallos de escritura |
| **Tratamiento de errores** | Cada rechazo tiene un código y se registra | Trazabilidad y métricas por tipo de fallo |

## 3. Firmware simulado (Fase 3)

El módulo `firmware_sim/device_sim.py` implementa la clase *DeviceSimulator*, que publica tres variables (temperatura ≈ 24 °C, humedad ≈ 55 %, voltaje ≈ 3,7 V, con ruido gaussiano), la identificación *device_id* y un contador *seq*. Permite detectar una condición anómala (temperatura entre 38 y 45 °C) e inyectar cuatro fallos de comunicación de forma determinista (por contador o por periodicidad):

- **badjson:** envía una línea que no es JSON válido.
- **corrupt:** altera un valor después de calcular el CRC, de modo que la integridad no coincide.
- **badrange:** envía temperatura de 999 °C con CRC correcto (dato íntegro pero fuera de rango).

- **stop/start:** cierra y reabre el servidor; el sensor sigue muestreando durante la caída, por lo que al volver la secuencia presenta un salto que permite medir la pérdida.

Esta separación entre *dato íntegro pero inválido* y *dato corrupto* permite probar por separado las capas de validación.

## 4. Aplicación Python modular (Fases 4 y 5)

| Módulo | Responsabilidad | Req. |
|---|---|---|
| **acquisition.py** | Cliente TCP con timeout; buffer acotado; separa líneas; lanza excepción en EOF o timeout | RF02, RF09 |
| **validation.py** | Valida estructura, tipo, CRC y rango. Códigos: BAD_FORMAT, MISSING_FIELD, BAD_TYPE, BAD_CRC, OUT_OF_RANGE | RF01, RF03 |
| **logic.py** | Clasifica NORMAL o ANOMALÍA (TEMP_ALTA, HUMEDAD_ALTA, VOLTAJE_BAJO) | RF04 |
| **os_info.py** | Consulta CPU, memoria, disco, PID y variable de entorno; evalúa umbrales de presión de recursos | RF06 |
| **persistence.py** | Escribe CSV abriendo el archivo en cada escritura; ante OSError guarda en un buffer y reintenta | RF07, RF09 |
| **report.py, main.py** | Orquesta, registra eventos, calcula indicadores, genera el reporte y muestra el estado | RF05, RF08, RF10 |

La validación se aplica en orden de costo creciente y falla en el primer defecto, devolviendo un código estable:

```
def parse_frame(line: bytes) -> dict:
    if len(line) > MAX_LINE: raise ValidationError("BAD_FORMAT", ...)
    obj = json.loads(line.decode("utf-8"))        # BAD_FORMAT si no es JSON
    # campos obligatorios -> MISSING_FIELD ; device_id (regex) y tipos -> BAD_TYPE
    payload = {k: v for k, v in obj.items() if k != "crc"}
    if obj["crc"] != _crc(payload): raise ValidationError("BAD_CRC", ...)
    # rangos temp [-40, 85], hum [0, 100], volt [0, 5.5] -> OUT_OF_RANGE
    return obj
```

El bucle principal no deja que ninguna excepción termine la aplicación: los timeouts y los cierres de conexión llevan al estado *RECONECTANDO*; las tramas rechazadas se cuentan y se registran; los errores de escritura pasan al buffer. El estado visible (RF10) es uno de CONECTADO, RECONECTANDO, DEGRADADO o FINALIZADO.

## 5. Integración con el sistema operativo (Fase 5)

| Recurso del SO | API usada | Relación con la operación |
|---|---|---|
| **Red** | socket (TCP) | Un EOF o un timeout indica pérdida del dispositivo y dispara la reconexión |
| **Archivos y permisos** | open, pathlib | Una ruta inaccesible genera OSError: buffer en memoria y vuelco al recuperarse |
| **CPU y memoria** | psutil | Por encima de 90 % el estado pasa a DEGRADADO y se registra RECURSOS_SO |
| **Disco** | shutil.disk_usage | Menos de 0,5 GB libres se informa como riesgo para la persistencia |
| **Proceso y entorno** | os.getpid, os.environ | PID y variable HSF_ENV se incluyen en el reporte para trazabilidad |

En cada fila del CSV se guardan además los porcentajes de CPU y memoria del momento, de modo que se pueda correlacionar la carga del sistema con la latencia y el tiempo de procesamiento observados.

## 6. Pruebas (Fase 6)

Las pruebas están automatizadas con pytest (`python -m pytest -q tests`), arrancan el simulador en un hilo y ejecutan la aplicación completa. Resultado: **9 pruebas aprobadas** en cerca de 4,5 s.

| ID | Prueba | Entrada / condición | Esperado | Real | Estado |
|---|---|---|---|---|---|
| TP-01 | Recepción periódica e identificación | 25 tramas normales | 25 válidas, ENV-001, seq creciente | 25 recibidas, 25 válidas | OK |
| TP-02 | Validación de trama | seq texto; device_id con comandos | BAD_TYPE / rechazo | BAD_TYPE y rechazo | OK |
| TP-03 | Normal vs. anómala | 4 anomalías en 30 tramas | 4 detectadas | 4 detectadas | OK |
| TP-04 | Recursos del SO | CPU, memoria, disco, PID, entorno | ≥ 2 recursos; alertas por umbral | 6 campos; alertas correctas | OK |
| TP-05 | CSV y reporte | 20 tramas | 20 filas, log y reporte | 20 filas, log y reporte | OK |
| TP-06 | Fallo: JSON incorrecto | 3 tramas rotas | 3 BAD_FORMAT; sigue | 3 rechazos; 27 válidas | OK |
| TP-07 | Fallo: CRC y rango | 2 CRC alterado; 2 con 999 °C | 2 BAD_CRC; 2 OUT_OF_RANGE | 2 y 2; 26 válidas | OK |
| TP-08 | Fallo: pérdida de comunicación | dispositivo detenido 1 s | reconexión < 5 s | recuperó en 1,4 s | OK |
| TP-09 | Fallo: ruta inaccesible | ruta no escribible y luego restaurada | buffer y vuelco sin pérdida | 2 filas, ninguna perdida | OK |

*Tabla 1. Matriz de pruebas (fuente: elaboración propia a partir de tests/test_integration.py).*

## 7. Fallos controlados

Se ejecutó un escenario de 160 tramas con fallos inyectados (caída de 1,5 s, JSON roto cada 29 tramas, CRC alterado cada 17, dato fuera de rango cada 41 y anomalías cada 20). La ruta sin permisos se verificó en TP-09.

| Fallo y causa | Síntoma | Detección | Respuesta del sistema | Evidencia | Propuesta de mejora |
|---|---|---|---|---|---|
| Caída del dispositivo (servidor detenido 1,5 s) | Dejan de llegar tramas | EOF y conexión rechazada | Estado RECONECTANDO; 4 reintentos con espera creciente | events.log: CONEXION_PERDIDA, REINTENTO, RECUPERACION 3,01 s | Limitar la espera máxima (p. ej. 1 s) y añadir jitter |
| Formato incorrecto (línea no JSON) | Trama ilegible | JSONDecodeError → BAD_FORMAT | Descarta y continúa | 5 de 160 tramas rechazadas | Alarma si la tasa supera un umbral |
| Dato corrupto o fuera de rango | Valores imposibles o CRC distinto | BAD_CRC y OUT_OF_RANGE | Rechaza; no se persiste | 9 BAD_CRC y 3 OUT_OF_RANGE | Confirmación/reenvío (ACK/NACK) |
| Ruta de escritura inaccesible | Error al guardar el CSV | OSError en persistence.py | Fila a buffer en memoria; vuelco al recuperarse | TP-09: 2 filas, 0 perdidas | Ruta de respaldo y alerta |

*Tabla 2. Fallos controlados (fuente: elaboración propia a partir de evidence/fallos).*

## 8. Observabilidad, indicadores y resultados (Fase 7)

Cada evento se registra con fecha y hora, nivel, evento y resultado. Fragmento real de `evidence/fallos/events.log`:

```
15:18:17,234 | WARNING | evento=TRAMA_RECHAZADA | resultado=BAD_CRC    | integridad no coincide
15:18:17,386 | WARNING | evento=ANOMALIA        | resultado=TEMP_ALTA  | ENV-001 seq=20 temp=44.13
15:18:18,448 | ERROR   | evento=CONEXION_PERDIDA | resultado=ConnectionError
15:18:19,852 | WARNING | evento=REINTENTO       | resultado=FALLIDO    | espera=1.6s
15:18:21,453 | INFO    | evento=CONEXION        | resultado=OK
15:18:21,454 | INFO    | evento=RECUPERACION    | resultado=OK         | 3.01s desde la deteccion
```

| Indicador | Cómo se mide | Meta | Corrida normal | Con fallos |
|---|---|---|---|---|
| **Disponibilidad de comunicación** | mensajes recibidos / esperados (por saltos de seq) | ≥ 95 % | 100 % (cumple) | 84,2 % (no cumple) |
| **Datos válidos** | datos aceptados / recibidos | ≥ 98 % | 100 % (cumple) | 89,4 % (no cumple) |
| **Tasa de errores** | errores / eventos | ≤ 5 % | 0 % (cumple) | 11,2 % (no cumple) |
| **Tiempo de recuperación** | detección de la caída → primera trama válida | ≤ 5 s | no aplica | 3,0 s (cumple) |
| **Latencia (media / p95)** | tiempo de recepción − tiempo de envío | por caso | 0,21 / 0,32 ms | 0,26 / 0,37 ms |
| **Procesamiento (media / p95)** | tiempo de validar, clasificar y guardar | por caso | 0,41 / 0,57 ms | 0,46 / 0,67 ms |

*Tabla 3. Indicadores técnicos (fuente: evidence/baseline/report.md y evidence/fallos/report.md).*

**Interpretación.** En la corrida normal (120 tramas, 6 anomalías de proceso, 0 rechazos) se cumplen todas las metas. En la corrida con fallos, 30 muestras se perdieron durante la caída (disponibilidad 84,2 %) y 17 de 160 tramas fueron rechazadas (89,4 % válidas): las metas no se cumplen porque los fallos se provocaron con una frecuencia muy superior a la real, pero ninguna trama defectuosa llegó al CSV (143 filas = 143 válidas). Los 3,0 s de recuperación superan el tiempo real de la caída (1,5 s) porque la espera creciente (0,2 → 1,6 s) hizo que el siguiente intento ocurriera después de que el dispositivo volviera; de ahí la mejora propuesta.

**Limitaciones.** La latencia usa el reloj del mismo equipo; con equipos distintos se requiere sincronización (NTP). La disponibilidad se estima por huecos de secuencia, por lo que no detecta pérdidas en el último tramo de una ejecución.

## 9. Requisitos no funcionales

| Requisito | Cómo se cumple |
|---|---|
| **Seguridad** | Validación de esquema, tipo, CRC y rango; device_id con expresión regular; límites de 4 KB y 64 KB; escucha solo en 127.0.0.1; no se usa subprocess ni shell. |
| **Confiabilidad** | Excepciones controladas; estados de desconexión; reconexión con espera creciente; buffer ante fallos de escritura. |
| **Rendimiento** | Se miden latencia y tiempo de procesamiento por trama (media, p95 y máximo). |
| **Mantenibilidad** | Cinco módulos con responsabilidad única: adquisición, validación, lógica, persistencia y presentación. |
| **Observabilidad** | Logs con fecha, nivel, evento y resultado; reporte con indicadores. |
| **Portabilidad** | pathlib para rutas; psutil con degradación a la biblioteca estándar; os.getloadavg solo existe en Linux; permisos (ACL o chmod) y puertos serie (COMx o /dev/ttyUSBx) difieren. |
| **Escalabilidad** | Pasar a un servidor concurrente (asyncio o broker MQTT) con una tarea por device_id, un almacén por dispositivo y métricas separadas. |

## 10. Respuestas a las preguntas de defensa técnica

### 1. ¿Qué responsabilidad es del hardware y cuál del firmware? ¿Dónde se transforma cada dato?

El hardware capta la señal física; el firmware muestrea, da formato y protege la trama (JSON + CRC32). El SO entrega los bytes por TCP y Python los valida, clasifica y persiste. La transformación ocurre en cada capa: señal → valor → trama → bytes → objeto → fila → reporte.

### 2. ¿Qué servicios del SO usa la aplicación?

Sockets TCP, sistema de archivos con permisos (CSV, log, reporte), procesos y entorno (PID, HSF_ENV) y consulta de CPU, memoria y disco.

### 3. ¿Qué ocurre si el SO revoca permisos sobre un recurso?

La escritura lanza OSError; se registra el evento ALMACENAMIENTO, la fila queda en un buffer (máx. 1000) y se vuelca al volver el permiso (TP-09). La adquisición no se detiene.

### 4. ¿Ventajas y limitaciones de Python frente a C/C++ o Rust?

Python ofrece desarrollo rápido, bibliotecas y pruebas sencillas. Su intérprete y recolector de basura impiden garantizar tiempo real duro y consumen más CPU y energía. Por eso el firmware se escribe en C o Rust y Python se reserva para el host.

### 5. ¿Qué debería ejecutarse en firmware y qué en software de aplicación?

En firmware: muestreo con temporización estricta, CRC y ahorro de energía. En la aplicación: reglas de negocio, clasificación, persistencia y reportes, que cambian con más frecuencia.

### 6. ¿Cómo cambia la arquitectura con 100 dispositivos simultáneos?

Se pasa a un servidor concurrente (asyncio o broker MQTT) con una tarea por dispositivo, cola de mensajes, base de series de tiempo y métricas por device_id.

### 7. ¿Qué riesgos aparecen al recibir datos externos o ejecutar comandos?

Inyección, tramas gigantes, denegación de servicio y suplantación. Se mitiga con validación, límites de tamaño, escucha local y ausencia de shell; en red, TLS y autenticación. Si hiciera falta ejecutar comandos: lista blanca y sin shell=True.

### 8. ¿Cómo garantizaría integridad y trazabilidad?

CRC32 por trama (para manipulación deliberada, HMAC). Cada fila lleva device_id, seq y marca de tiempo; los huecos de seq se miden y el log registra fecha, nivel, evento y resultado. Mejora: hash SHA-256 del CSV al cerrar.

### 9. ¿Diferencias entre Windows y Linux?

Rutas (resueltas con pathlib), permisos (ACL o chmod), puertos serie (COMx o /dev/ttyUSBx), load average solo en Linux y administración de servicios (Servicio de Windows o systemd).

### 10. ¿Cómo actualizar el firmware sin afectar el servicio?

Doble banco A/B con reversión automática, imagen firmada, despliegue por oleadas, versión de firmware en la trama y protocolo compatible hacia atrás; la aplicación tolera el reinicio reconectando.

### 11. ¿Qué pruebas adicionales antes de producción?

Carga con 100 o más dispositivos, larga duración, fuzzing de tramas, pruebas de seguridad, disco lleno, hardware real, y CI con pytest en Windows y Linux.

### 12. ¿Cuál sería el principal punto de falla y cómo hacerlo tolerante?

El enlace único y la aplicación como proceso único. Se mitiga con buffer en el firmware, reconexión con jitter, supervisor (systemd o watchdog) y un colector redundante.

## 11. Conclusiones

- La integración funciona de extremo a extremo y cubre los requisitos RF01 a RF10 con 9 pruebas automatizadas aprobadas.
- Validar por capas (estructura, tipo, CRC, rango) evitó que alguna de las 17 tramas defectuosas llegara al CSV.
- La resiliencia debe medirse: el sistema se recuperó en 3,0 s de una caída de 1,5 s, y la política de espera creciente es el punto a ajustar.
- Siguientes pasos: probar con una placa real (por ejemplo ESP32), escalar con MQTT y llevar el firmware a Rust para cumplir tiempo real y reducir consumo.

## Referencias

Marwedel, P. (2021). *Embedded system design: Embedded systems foundations of cyber-physical systems, and the Internet of Things* (4.ª ed.). Springer.

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

Python Software Foundation. (s. f.). *logging — Logging facility for Python*. Python 3 documentation. https://docs.python.org/3/library/logging.html

Python Software Foundation. (s. f.). *socket — Low-level networking interface*. Python 3 documentation. https://docs.python.org/3/library/socket.html

Rodola, G. (s. f.). *psutil documentation*. https://psutil.readthedocs.io/

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10.ª ed.). Wiley.

Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4.ª ed.). Pearson.

## Anexo. Reproducción

```
pip install -r requirements.txt
python -m pytest -q tests      # 9 passed
python run_demo.py             # genera evidence/baseline y evidence/fallos
```